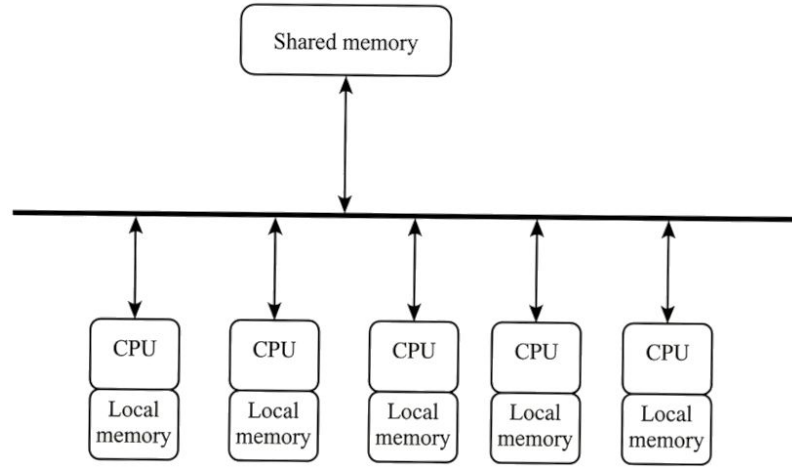


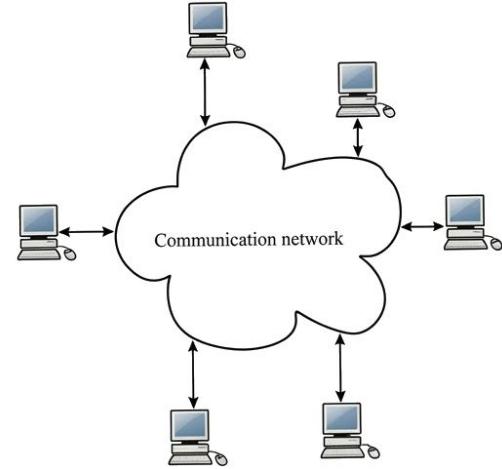
Distribuirani računarski sistemi

Distribuirano programiranje

Paralelni i distribuirani sistemi



Paralelni sistem



Distribuirani sistem

Prednosti distribuiranog računarskog sistema

- Skalabilnost
- Modularnost i heterogenost
- Dijeljenje i geografska distribucija podataka
- Dijeljenje resursa
- Pouzdanost
- Cijena

Zašto ne koristiti uvijek distribuirani računarski sistem?

- Zato što je brže da se ažurira dijeljena memorijska lokacija nego da se pošalje poruka preko mreže.

Komunikacija između procesa

- Sockets

- Komunikacija se zasniva na slanju sirovih bajtova podataka između IP adresa i portova.
- Programer mora definisati sopstveni protokol (format poruke), vršiti serijalizaciju (pretvaranje objekta u bajtove) i ručno upravljati konekcijom.
- Pružaju veću fleksibilnost, možeš komunicirati sa bilo kojim jezikom (npr. Java klijent priča sa C++ serverom).

- Remote Method Invocations (RMI)

- **specifično Java** rješenje omogućava da objekat na jednom računaru pozove metodu na objektu koji se nalazi na potpuno drugom računaru
- Programer ne vidi mrežne detalje (IP adrese, portove, stream-ove). Sve izgleda kao standardno objektno-orijentisano programiranje.

IP adrese

- Svaki računar u mreži mora imati jedinstvenu adresu koja služi za identifikaciju računara
- Hostname (npr. [google.com](https://www.google.com)) predstavlja je logičko ime koje ljudi lako čitaju i pamte.
- DNS je servis prevodi hostname u ip adresu.
- `Java.net.InetAddress` - ugrađena klasa za rad sa adresama.
 - `public byte[] getAddress()`
 - Returns the raw IP address of this `InetAddress` object.
 - `public static InetAddress getByName(String host)`
 - Determines the IP address of a host, given the host's name.
 - `public String.getHostAddress()`
 - Returns the IP address string "%d.%d.%d.%d".
 - `public String.getHostName()`
 - Returns the fully qualified host name for this address.
 - `public static InetAddress getLocalHost()`
 - Returns the local host.

Soketi TCP/UDP

UDP (User Datagram Protocol) – connectionless

- **Nema garancije:** Paketi mogu stići van reda, ili se mogu potpuno izgubiti u mreži.
- **Bez potvrde:** Ne postoji "handshaking" (rukovanje) niti potvrda prijema (acknowledgments).
- **Prednost:** Izuzetno je efikasan jer nema "overhead-a" (dodatnih podataka za provjeru grešaka).
- **Primjena:** Video streaming, online igre, VOIP – svuda gdje je bitnije da podatak stigne brzo nego da je svaki frejm savršen.

TCP (Transmission Control Protocol) – connection-oriented

- **Garantovana dostava:** Svaki paket mora biti potvrđen. Ako se izgubi, šalje se ponovo.
- **Redoslijed:** Paketi stižu tačno onim redom kojim su poslali.
- **Mana:** Manje je efikasan od UDP-a jer troši vrijeme i resurse na provjeru i uspostavljanje veze.
- **Primjena:** Web (HTTP), Email, prenos fajlova (FTP) – svuda gdje gubitak jednog bajta kvari cijelu poruku.

UDP Soketi

- Realizacija UDP komunikacije u Javi
- Individualno adresiranje - svaki paket se posebno adresira. Svaki paket nosi informaciju o tome kome ide (IP adresa i port).
- Dva paketa poslata jedan za drugim mogu putovati različitim putevima kroz internet.
- Zbog toga mogu stići bilo kojim redoslijedom (drugi može stići prije prvog).
- Prednost: Brzina
- `new DatagramSocket()` - sistem mu sam dodijeli slobodan port.
- `new DatagramSocket(1234)`
- `new DatagramSocket(int, InetAddress laddr)`

Metode klase DatagramSocket

- `close()`
 - Zatvara datagram soket i oslobađa resurse.
- `getLocalPort()`
 - Vraća broj porta na lokalnom hostu na koji je soket povezan.
- `getLocalAddress()`
 - Vraća lokalnu adresu (`InetAddress`) na koju je soket povezan.
- `receive(DatagramPacket p)`
 - Prima paket sa soketa.
 - Blokira izvršavanje dok paket ne stigne (osim ako je postavljen timeout).
 - Popunjava bafer paketa podacima, IP adresom i portom pošiljaoca.
- `send(DatagramPacket p)`
 - Šalje paket sa ovog soketa.
 - Paket mora sadržati podatke, dužinu, IP adresu i port odredišta.

Datagram paketi

- Da bi se podaci poslali preko DatagramSocket-a, oni moraju biti upakovani u objekte klase `java.net.DatagramPacket`.
- `DatagramPacket` sadrži:
 - Podaci (Payload): Niz bajtova (`byte[]`) koji se šalje ili prima.
 - Dužina (Length): Koliko je bajtova iz niza zapravo iskorišćeno za poruku.
 - Adresa (IP Address):
 - Pri slanju: Adresa primaoca.
 - Pri prijemu: Adresa pošiljaoca.
 - Port:
 - Pri slanju: Port na udaljenoj mašini.
 - Pri prijemu: Port sa kojeg je paket poslat.

TCP Soketi

- Connection-oriented: Mora se uspostaviti veza između pošiljaoca i primaoca pre slanja podataka.
- Garantovana isporuka: Protokol se brine da svaki bajt stigne na odredište.
- Očuvanje redosleda: Paketi se primaju tačno onim redosledom kojim su poslali.
- Error Recovery: Napredniji mehanizmi za oporavak od grešaka u prenosu.

TCP Soketi

- Klasa `java.net.Socket` se koristi za TCP komunikaciju u Javi na klijentskoj strani
- `public Socket(String host, int port)`
 - `host`: Ime servera (npr. `"localhost"` ili `"www.google.com"`) ili IP adresa.
 - `port`: Broj porta na kojem server sluša.
- Napomena: Ovaj konstruktor automatski pokušava da uspostavi vezu. Ako server nije dostupan, baca se `IOException`.

TCP Soketi

- Klasa `java.net.ServerSocket` se koristi za TCP komunikaciju u Javi na serverskoj strani
- Instanciranje: `ServerSocket server = new ServerSocket(9876);`
- Čekanje: Poziv `Socket klijent = server.accept();` (Server ovde stoji).
- Komunikacija: Koriste se `InputStream` i `OutputStream` dobijeni iz klijent soketa.
- Zatvaranje: Zatvaraju se tokovi i soketi nakon završetka posla.

Primjer: Linker

Napisati klasu Linker, koja omogućava komunikaciju između skupa procesa P_1 , $P_2 \dots P_n$.

Potrebno je da svaki proces može da šalje poruke svim ostalim procesima i da prima poruke od svih ostalih procesa

Java Remote Method Invocation (RMI)

- API koji omogućava da objekat na jednoj Java Virtual Machine (JVM) pozove metodu objekta na drugoj JVM, moguće i na drugom hostu
- Proces koji kreira poziv predstavlja klijenta
- Proces na kojem se nalazi objekat čija se metoda poziva zove se server
- U pozadini, naziv metoda i argumenti se upakuju u poruku koja se šalje serveru.
- U pozadini se odvija TCP komunikacija
- Server nakon izvršavanja metoda upakuje rezultat u poruku, i pošalje je
- Specifično za JVM, ne radi sa ostalim programskim jezicima
- Performanse, sporije od socketa

Terminologija

1. Stub (Klijentska strana)

- a. Stub je zastupnik (proxy) udaljenog objekta koji se nalazi na klijentu.
- b. Šta radi: Kada klijent pozove metodu, on zapravo poziva metodu na Stubu.
- c. Zadatak: Stub pakuje podatke (ovaj proces se zove Marshalling) i šalje ih preko mreže serveru. On "glumi" da je pravi objekat kako bi klijentski kod bio jednostavan.

2. Skeleton (Serverska strana)

- a. Skeleton je pomoćni objekat na strani servera.
- b. Šta radi: On prima mrežni zahtjev od Stuba.
- c. Zadatak: On raspakuje podatke (Unmarshalling), poziva pravu metodu na stvarnom objektu na serveru, uzima rezultat, ponovo ga pakuje i šalje nazad Stubu.

3. U novijim verzijama jave, ne vidite Skeleton fajlove, a Stub se generiše automatski u memoriji.

Remote objekat

Remote objekat je objekat koji se može pozvati iz druge JVM.

Objekat je opisan interfejsom. Interfejs mora da nasleđuje interfejs Remote.

Sve metode unutar interfejsa moraju da bacaju izuzetak tipa RemoteException.

Remote interface mora da bude dostupan i na klijentskoj i na serverskoj strani.

Na serverskoj strani treba implementirati i klasu koja implementira interfejs i nasljedjuje klasu `UnicastRemoteObject`

Java Remote Method Invocation (RMI)

- Prilikom pozivanja metoda
 - Argumenti se prenose kao poruke preko mreže
 - Povratna vrijednost se prenosi kao poruka preko mreže

Prenos objekata

- Primitivni tipovi se prenose po vrijednosti
- Lokalni objekti se šalju tako što se izvrši serijalizacija (pretvaranje objekta u niz bitova) i pošalje se serijalizovana verzija. Objekat treba da implementira interfejs Serializable.
- Ako objekat koji serijalizujemo sadrži reference na druge objekte, onda se i svi ti objekti moraju serijalizovati.
- Kao rezultat poziva metode preko RMI, dobija se lokalni objekat.
- Reference na objekte koji implementiraju Remote interfejs se predaju kao remote reference. (Stub za remote objekt se predaje)

Klijent

- Klijent treba da dobijer refernecu na Remote objekat
- Klasa `java.rmi.Naming` služi za uzimanje reference na remote objekat
- Referenca se dobija na osnovu Uniform Resource Locator (URL)-a
- URL je oblika:
 - `rmi://host:port/name`
 - `host` - host name na kojem se nalazi registry
 - `port` - port na kojem sluša registry
 - `name` - ime remote objekta

Sinhronizacija

Izvršavanje programa se može posmatrati kao niz događaja (events).

Primjeri su:

- Početak ili kraj neke funkcije.
- Slanje poruke sa jednog računara.
- Prijem poruke na drugom računaru.

Samo postojanje ovih događaja nije dovoljno; moramo znati kojim su se redosledom desili da bismo razumjeli kako softver radi.

Balance = 130

P1: read(balance) $\leq$ 100

P2: read(balance) $\leq$ 100

P1: balance = balance - 50 = 50

P2: balance = balance + 30 = 130

P1: write(balance)

P2: write(balance)

Interleaving model (Model ispreplitanja)

Ovaj model se oslanja na fizičko vreme.

Pretpostavka: Postoji jedan savršen, zajednički sat koji svi računari vide.

Rezultat: Svaki događaj ima tačno vreme. Čak i ako se dese u deliću sekunde razmaka, možemo ih poređati u jednu pravu liniju (totalni poredak).

Problem: U stvarnom svetu, računari u mreži retko imaju savršeno sinhronizovane satove.

Happened-before model (Model "desilo se pre")

Ovo je realističniji model za sisteme bez zajedničkog sata.

Logika: Redosled određujemo na osnovu protoka informacija. Ako je procesor A poslao poruku, a procesor B je primio, znamo da se slanje desilo pre prijema.

Rezultat: Imamo parcijalni poredak. Znamo redosled događaja na istom procesoru, ali za dva događaja na različitim procesorima (koji ne komuniciraju) možda ne možemo sa sigurnošću reći koji je bio prvi.

Logički satovi

Mehanizmi (algoritmi) koji dodjeljuju brojeve događajima tako da možemo da napravimo "vještački" totalni poredak koji je logički ispravan, čak i ako se ne poklapa savršeno sa stvarnim vremenom u nanosekundama.

Svojstva distribuiranog sistema

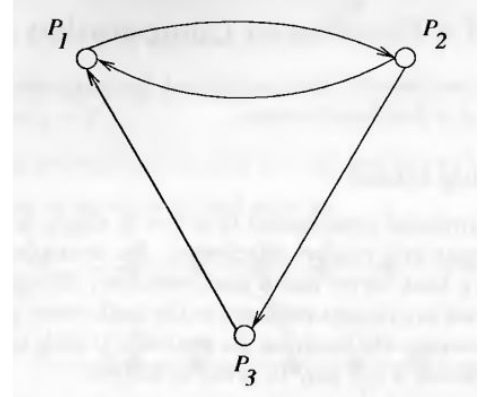
- Ne postoji dijeljeni časovnik: Nije moguće precizno sinhronizovati časovnike na različitim procesorima precizno
 - Komunikacija preko mreže unosi kašnjenje koje nije fiksno
 - Ne možemo se osloniti na fizičko vrijeme za sinhronizaciju procesa u distribuiranom sistemu
- Ne postoji dijeljena memorija
 - Ni jedan proces ne može da zna globalno stanje sistema
- Nedostatak pouzdane detekcije kvarova
 - Ne možemo da znamo da li izvršavanje na nekom procesu traje previše dugo zato što je procesor spor, ili je došlo do kvara i procesor nikada neće poslati odgovor

Pretpostavićemo da se komunikacija među procesima odvija slanjem poruka.

Pretpostavićemo da neki od parova čvorova mogu direktno da komuniciraju.

Svojstva distribuiranog sistema

- Pretpostavljamo da kanali komunikacije imaju buffer beskonačne veličine
- Pretpostavićemo da se ne dešavaju greške u komunikaciji
- Ne pravimo pretpostavke o vremenu potrebnom da se pošalje poruka niti o redosljedu kojim poruke stižu.



Sinhronizacija na centralizovanom vs distribuiranom sistemu

- Kod centralizovanog sistema svi procesi koriste zajednički časovnik
 - Lako je odrediti redosljed događaja
- U distribuiranom sistemu svaki CPU ima svoj časovnik
 - Teža sinhronizacija

Sinhronizacija na centralizovanom vs distribuiranom sistemu

Procesor A (10:00:01): Korisnik je kupio proizvod.

Procesor B (10:00:00) - kasni mu sat: Korisnik otkazuje kupovinu.

??Kako korisnik otkazuje kupovinu kad još nije kupio proizvod??

Happened before model

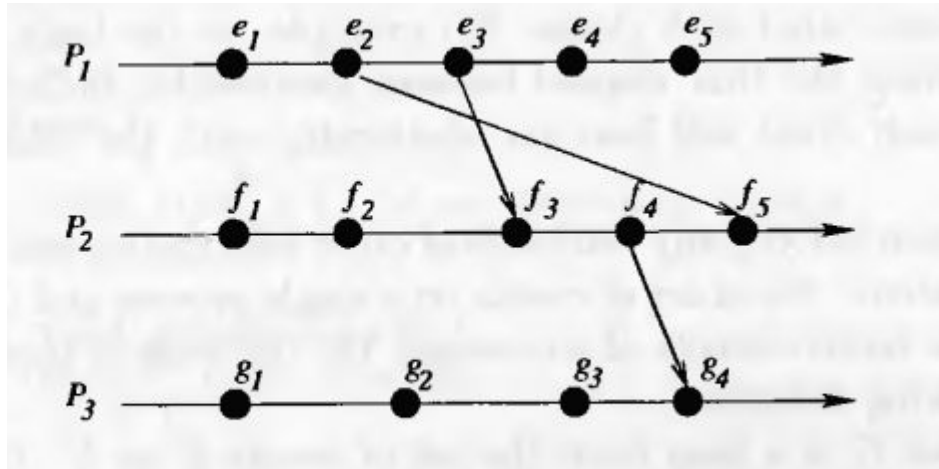
- Ako dva računara nemaju interakcija, nema potrebe da ih sinhronizujemo
- Fizičko vrijeme u kojem se desio neki događaj nije bitno. Bitno je samo da se procesi usaglase kojim redoslijedom su se desili događaji

Happened before model

Happened before ($\rightarrow$) relacija je najmanja relacija koja zadovoljava:

1. Ako se je na nekom procesu događaj e dogodio prije događaja f , tada važi $e \rightarrow f$.
2. Ako je e događaj poruka m je poslata, a f događaj poruka m je primljena, tada važi $e \rightarrow f$.
3. Ako postoji događaj g , takav da $(e \rightarrow g)$ i $(g \rightarrow f)$, tada važi da $(e \rightarrow f)$.

Happened before model



Happened before diagram

$e_2 \rightarrow e_4, e_3 \rightarrow f_3, e_1 \rightarrow g_4$

Happened before model

- Happened before relacija predstavlja parcijalni poredak na skupu događaja
- Moguće je da ne možemo pomoću relacija happened before da uspostavimo poredak između neka dva događaja
- Kažemo da su događaji e i f konkurentni ($e \parallel f$) ako važi:

$$\neg(e \rightarrow f) \wedge \neg(f \rightarrow e).$$

Logical clocks

- Prati relaciju Happened before model
- Cilj je da uspostavimo redosljed među događajima, ne i da uspostavimo fizičko mjerenje vremena
- Logički sat C je preslikavanje iz skupa događaja E u skup prirodnih brojeva N sa svojstvom

$$\forall e, f \in E : e \rightarrow f \Rightarrow C(e) < C(f)$$

- Potpuni poredak može da se uspostavi na skupu događaja ako posmatramo i proces kojem događaj pripada

$$(s.c, s.p) < (t.c, t.p) \stackrel{\text{def}}{=} (s.c < t.c) \vee ((s.c = t.c) \wedge (s.p < t.p)).$$

Lamportov časovník

```
1 public class LamportClock {
2     int c;
3     public LamportClock () {
4         c = 1;
5     }
6     public int getValue () {
7         return c;
8     }
9     public void tick () { // on internal events
10        c = c + 1;
11    }
12    public void sendAction () {
13        // include c in message
14        c = c + 1;
15    }
16    public void receiveAction(int src, int sentValue) {
17        c = Util.max(c, sentValue) + 1;
18    }
19 }
```

Primjer

Zamislamo troje studenata: **A (Marko)**, **B (Janko)** i **C (Ivana)** u grupnom chatu.

1. **Marko (A)** šalje poruku: *"Hoćemo li na kafu?"*
2. **Janko (B)** prima Markovu poruku i odgovara: *"Može, u 5h."*
3. **Ivana (C)** je bila offline. Kada se uključi, njena mreža je spora.

Problem sa Lamportovim časovnikom:

Zbog mrežnog kašnjenja, Ivani se može desiti da prvo stigne Jankova poruka (*"Može, u 5h"*), a tek onda Markova (*"Hoćemo li na kafu?"*).

Vector clocks

- Ako važi $e \rightarrow f$, tada za Lamportov sat važi da je $C(e) < C(f)$.
- Međutim ako važi da je $C(e) < C(f)$, ne mora nužno da važi da je $e \rightarrow f$.
- Ne možemo da detektujemo konkurentnost.
- Lamportov sat pokušava sve događaje da prikaže u jednoj vremenskoj liniji.
- Primjer:
 - a. Korisnik 1 mjenja polje u tabeli na serveru u Evropi (Sat: 10).
 - b. Korisnik 2 mjenja polje u tabeli na serveru u Americi (Sat: 15).
- Sistem koji koristi Lamportov sat će pogledati ove brojeve i reći: "Pošto je 10 manje od 15, a se sigurno desilo prije b ." To je greška! Događaji a i b su se desili nezavisno (konkurentno). Proces B nije imao pojma šta je Proces A radio. Oni su u konfliktu jer su oba mjenjala istu stvar bez međusobne sinhronizacije, ali Lamportov sat nam daje lažnu sliku da je postojao jasan redosljed.

Vector clocks

- Vektorski sat C je preslikavanje iz skupa događaja E u skup N^k sa svojstvom

$$\forall s, t : s \rightarrow t \Leftrightarrow s.v < t.v.$$

- Gdje je $s.v$ vektor dodijeljen stanju s .
- Poređenje vektora se vrši na sljedeći način:

$$\begin{aligned} x < y &= (\forall k : 1 \leq k \leq N : x[k] \leq y[k]) \wedge \\ &\quad (\exists j : 1 \leq j \leq N : x[j] < y[j]) \\ x \leq y &= (x < y) \vee (x = y) \end{aligned}$$

Vektor clocks

```
1 public class VectorClock {  
2     public int [] v;  
3     int myId;  
4     int N;  
5     public VectorClock(int numProc, int id) {  
6         myId = id;  
7         N = numProc;  
8         v = new int[numProc];  
9         for (int i = 0; i < N; i++) v[i] = 0;  
10        v[myId] = 1;  
11    }  
12    public void tick() {  
13        v[myId]++;  
14    }  
15    public void sendAction() {  
16        //include the vector in the message  
17        v[myId]++;  
18    }  
19    public void receiveAction(int [] sentValue) {  
20        for (int i = 0; i < N; i++)  
21            v[i] = Util.max(v[i], sentValue[i]);  
22        v[myId]++;  
23    }  
24    public int getValue(int i) {  
25        return v[i];  
26    }  
27    public String toString(){  
28        return Util.writeArray(v);  
29    }  
30 }
```

Primjer

Korisnik dodaje artikle u korpu sa svog telefona (Čvor A) i laptopa (Čvor B). Sistem koristi vektorske časovnike $[V_A, V_B]$.

1. Korisnik na **telefonu (A)** doda "Majicu". Stanje: {Majica}, Vektor: [1, 0].
2. Telefon se sinhronizuje sa **laptopom (B)**. Laptop sada ima: {Majica}, Vektor: [1, 0].
3. Korisnik na **laptopu (B)** doda "Pantalone". Stanje: {Majica, Pantalone}, Vektor: [1, 1].
4. U istom trenutku, dok je telefon offline, korisnik na **telefonu (A)** obriše "Majicu" i doda "Šorts". Stanje: {Šorts}, Vektor: [2, 0].

Primjer

Dva studenta, **P1** i **P2**, edituju zajedničku bilješku sa predavanja na Google Drive-u. Koriste vektorske časovnike $[V_1, V_2]$ za detekciju konflikata. Početno stanje je $[0, 0]$

1. **P1** unese izmjenu i pošalje verziju sa vektorom $[1, 0]$.
2. **P2** (ne znajući za izmjenu od P1) unese svoju izmjenu i pošalje verziju sa vektorom $[0, 1]$.
3. **Server** prima obje verzije.

Pitanja:

- a) Uporedite vektore $[1, 0]$ i $[0, 1]$. Da li je jedan manji od drugog?
- b) Šta server zaključuje na osnovu ove neuporedivosti?
- c) Kako bi vektori izgledali da je P2 prvo primio izmjenu od P1, pa tek onda dodao svoju?

Mutual exclusion

Koordinacija među procesima.

- Mutual exclusion
- Event ordering

Ako imamo neku kritičnu sekciju:

- Safety - Dva procesa ne mogu biti u kritičnoj sekciji
- Liveness - Svaki zahtjev za ulazak u kritičnu sekciju će biti ispunjen u nekom trenutku
- Pravednost - Zahtjevi bi trebali biti ispunjeni redoslijedom kojim su kreirani

Primjer 1

Podizanje novca sa računa

Pretpostavimo da je početno stanje na računu: **100 €**.

1. **Proces A** : Želi da podigne **80 €**.
2. **Proces B** : Želi da podigne **50 €**.

Dozvoljene operacije su:

READ(STANEJE)

WRITE(STANJE)

Bez Mutual Exclusion-a

P1: Proces A očita stanje: **100 €**.

P2: Proces B očita stanje: **100 €**.

P3: Proces A provjeri: $100 > 80$, odobri transakciju, oduzme novac i upiše novo stanje: **20 €**.

P4: Proces B provjeri: $100 > 50$ (koristi staru vrijednost), odobri transakciju, oduzme novac i upiše novo stanje: **50 €**.

Rezultat: Banka je isplatila ukupno **130 €**, a na računu je ostalo **50 €**, iako je trebalo da bude u minusu ili da druga transakcija bude odbijena. Ovo se zove **Race Condition**.

P1: Zelim da udjem u KS

P2: Zelim da udjem u KS

Kordinator: P1 moze da udje

P1: Proces A očitá stanje: 100 €.

P1: Proces A provjeri: $100 > 80$, odobri transakciju, oduzme novac i upiše novo stanje: 20 €.

P1: Oslobodi KS 1

Koordinator: P2 moze da udje u KS

P2: Proces B očitá stanje: 100 €.

P2: Proces B provjeri: $100 > 50$ (koristi staru vrijednost), odobri transakciju, oduzme novac i upiše novo stanje: 50 €.

P2: Oslobodi KS 1

Rezultat: Banka je isplatila ukupno **130 €**, a na računu je ostalo **50 €**, iako je trebalo da bude u minusu ili da druga transakcija bude odbijena. Ovo se zove **Race Condition**.

Mutual exclusion

- Pristup dijeljenim resursima
 - Timestamp based algoritmi
 - Token based algoritmi
 - Algoritmi zasnovani na kvorumu

Centralizovani algoritam za mutual exclusion

- Jedan od procesa predstavlja koordinatora kritične sekcije
- Svaki proces koji želi da uđe u kritičnu sekciju, šalje zahtjev koordinatoru
- Koordinator smješta zahtjeve u red za čekanje
- Kad proces stigne na vrh reda, dobije od koordinatora dozvolu da uđe u kritičnu sekciju.
- Kad proces završi rad u kritičnoj sekciji šalje poruku koordinatoru da su resursi oslobođeni.

Centralizovani algoritam

```
public class CentMutex extends Process implements Lock {
    // assumes that P_0 coordinates and does not request locks.
    boolean haveToken;
    final int leader = 0;
    IntLinkedList pendingQ = new IntLinkedList();
    public CentMutex(Linker initComm) {
        super(initComm);
        haveToken = (myId == leader);
    }
    public synchronized void requestCS() {
        sendMsg(leader, "request");
        while (!haveToken) myWait();
    }
    public synchronized void releaseCS() {
        sendMsg(leader, "release");
        haveToken = false;
    }
    public synchronized void handleMsg(Msg m, int src, String tag) {
        if (tag.equals("request")) {
            if (haveToken) {
                sendMsg(src, "okay");
                haveToken = false;
            }
            else
                pendingQ.add(src);
        } else if (tag.equals("release")) {
            if (!pendingQ.isEmpty()) {
                int pid = pendingQ.removeHead();
                sendMsg(pid, "okay");
            } else
                haveToken = true;
        } else if (tag.equals("okay")) {
            haveToken = true;
            notify();
        }
    }
}
```

Redosljed ulaska u kritičnu sekciju

Redosljed ulaska u kritičnu sekciju ne mora da bude onim redosledom kojim su zahtjevi napravljeni.

Imamo sistem sa dva procesa (P_1 , P_2) i jednim centralnim koordinatorom (C).

1. Proces P_1 šalje zahtjev REQUEST koordinatoru. (50ms dok stigne)
2. Proces P_2 šalje svoj REQUEST. Mrežni prenos: (2ms dok stigne).
3. Obrada koordinatora:
 - a. Koordinator prvo prima poruku od P_2 .
 - b. On vidi da je kritična sekcija slobodna i odmah šalje GRANT procesu P_2 .
 - c. Nakon 48 ms, koordinatoru stiže poruka od P_1 .
 - d. Koordinator ga stavlja u red čekanja.

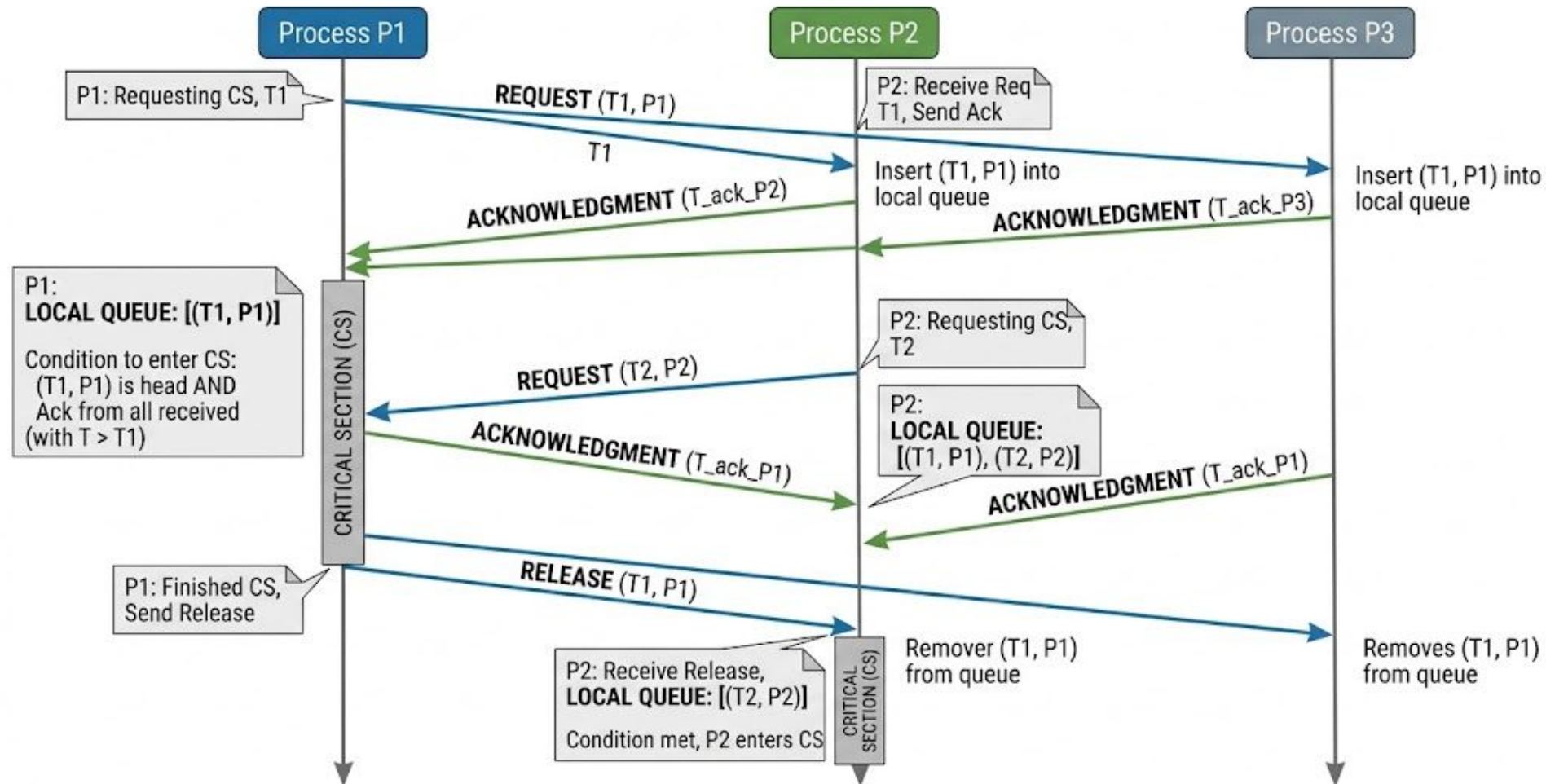
Lamportov algoritam za mutual exclusion

- Koristi Lamportove logičke satove kako bi se uspostavio totalni poredak
- Svaki proces P_i održava lokalni red čekanja (request queue) u koji smešta zahteve za resursom, sortirane prema timestamp-u.
- Da bi se izbjegle kolizije kada dva procesa imaju isti timestamp, koristi se identifikator procesa kao "tie-breaker":

Lamportov algoritam za mutual exclusion

1. Slanje zahteva: Kada proces P_i želi da pristupi resursu:
 - a. Šalje poruku REQUEST(T_i, P_i) svim ostalim procesima.
 - b. Postavlja taj isti zahtev u svoj lokalni red čekanja.
2. Prijem zahteva: Kada proces P_j primi poruku REQUEST(T_i, P_i):
 - a. Šalje nazad poruku ACKNOWLEDGMENT procesu P_i .
 - b. Stavlja zahtev (T_i, P_i) u svoj lokalni red čekanja.
3. Ulazak u kritičnu sekciju: Proces P_i ulazi u kritičnu sekciju tek kada su ispunjena dva uslova:
 - a. Njegov sopstveni zahtev (T_i, P_i) je na vrhu njegovog lokalnog reda.
 - b. Primio je potvrde (ACK) od svih ostalih procesa sa timestamp-om većim od T_i .
4. Oslobađanje resursa: Kada P_i završi rad sa resursom:
 - a. Uklanja svoj zahtev iz lokalnog reda.
 - b. Šalje poruku RELEASE svim ostalim procesima.
 - c. Kada ostali procesi prime RELEASE, oni uklanjaju zahtev P_i iz svojih redova, što omogućava sledećem procesu na vrhu reda da dobije pristup.

Lamport's Distributed Mutual Exclusion Algorithm - Resource Allocation Diagram



Karakteristike Lamportovog algoritma

- Broj poruka - Za svaki ulazak u kritičnu sekciju potrebno je $3(N-1)$ poruka
 - $N-1$ - poruka zahtjeva
 - $N-1$ - potvrda
 - $N-1$ - oslobađanja
- Pouzdanost - Zahijteva pouzdan komunikacioni kanal gde se poruke ne gube i stižu u redosledu slanja (FIFO).
- Decentralizacija - Ne postoji centralni koordinator; odluka je distribuirana.

Tačka otkaza

- **Centralizovani:** Ako koordinator padne, sistem je blokiran. Niko ne može dobiti resurs dok se koordinator ne oporavi ili se ne izabere novi.
- **Lamportov:** Nema centralne tačke otkaza. Lamportov algoritam je osjetljiv na pad **bilo kog** procesa.

Kašnjenje

- **Centralizovani:** Minimalno. Jedan zahtev ide do koordinatora, on odgovara čim je resurs slobodan.
- **Lamportov:** Postoji veće kašnjenje jer proces mora da komunicira sa svakim čvorom u sistemu prije nego što bude siguran da on ima najraniji timestamp.

Rješenje uz Mutual Exclusion:

Proces A šalje zahtjev za pristup resursu "Stanje računa".

Proces A dobija dozvolu i **zaključava** resurs (ulazi u Kritičnu Sekciju).

Proces B šalje zahtjev, ali on biva stavljen u red čekanja.

Proces A završi sve: očita 100, isplati 80, upiše 20.

Proces A oslobađa resurs.

Proces B tek sada dobija dozvolu, očitava **ново stanje (20 €)**, vidi da nema dovoljno novca ($20 < 50$) i transakcija se odbija.

Primjeri

Avio-kompanija koristi globalnu mrežu od N servera. Svaki klijent pristupa najbližem serveru kako bi rezervisao sjedište. Možete pretpostaviti da imate jedan server koji predstavlja bazu podataka. Za potrebe zadatka možemo pretpostaviti da server čuva listu već bookiranih sjedišta. Bookirano sjedište je opisano strukturom Ticket { String brojLeta; String brojSjedista; String JMBG; }.

- Imate N server koji opslužuju klijentske zahtjeve iz različitih dijelova svijeta. Klijentski server mogu da serveru sa bazom podataka pošalju dva zahtjeva, isAvailable(String brojLeta, String brojSjedista) i book(String brojLeta, String brojSjedišta, String JMBG).
- Komunikacija između servera sa bazom podataka i servera koji opslužuju klijentske zahtjeve se odvija preko TCP Soketa.
- Da bi se spriječilo da dva klijenta rezervišu isto sjedište u isto vrijeme, koriste **Centralizovani/Lamportov** algoritam.

Primjeri

Dizajnirate pojednostavljenu verziju sistema nalik na Google Drive. Sistem se sastoji od **N klijentskih čvorova** (računara korisnika) i jednog **Centralnog File Servera**.

- **Centralni Server:** Čuva fajlove i metapodatke. Svaki fajl je opisan strukturom: `struct FileMetadata { string fileName; long fileSize; string lastModifiedBy; string contentHash; }`.
- **Klijentski Čvorovi:** Lokalno edituju fajlove. Da bi sačuvali izmjene na serveru, klijenti koriste TCP sokete i dvije operacije:
 1. `downloadForEdit(string fileName)` – preuzima fajl i postavlja lokalni lock.
 2. `uploadChanges(string fileName, string newContent)` – šalje izmjene i oslobađa resurs.

Problem: Kada više korisnika želi da edituje isti fajl (npr. projekat.txt), moramo osigurati da samo jedan korisnik ima "pravo pisanja" u datom trenutku. Za koordinaciju između klijentskih čvorova koristi se **Lamportov/Centralizovani algoritam**.

Primjeri

Dizajnirate sistem za distribuirano učenje. Sistem se sastoji od jednog **Centralnog Servera (Glavni Model)** i **N Radnih Čvorova (Workers)**.

1. **Radni čvorovi** preuzimaju trenutno stanje modela, vrše proračun na svojim podacima i generišu **predlog izmjena** (gradijente).
2. **Centralni Server** prima te izmjene i primjenjuje ih na model.

Problem sinhronizacije: Da bi učenje bilo ispravno, Centralni Server mora da radi u **koracima (iteracijama)**. On ne smije da prihvati izmjene za "Korak 2" dok god svi radni čvorovi nisu poslali svoje izmjene za "Korak 1". Ako neki čvor zbog bržeg procesora pokuša da pošalje izmjene unaprijed, on mora biti zaustavljen.

Za regulisanje ovog reda čekanja koristi se **Lamportov algoritam**.

Kraj